



JavaRunner - Manual de usuario

Traductores

Docente: Guillermo Licea Sandoval

Alumnos:

- Chaparro Herrera Hugo Giovanni - 2200357
- Rivera Vazquez Hugo Alexis - 2200073

Tijuana, Baja California a 28 de mayo de 2026



Índice

Introducción.....	3
Requisitos del sistema.....	3
Sintaxis y características permitidas.....	3
Tipos de Datos y Variables.....	3
Operadores.....	3
Estructuras de Control.....	4
Salida estándar.....	4
Guia de uso y ejecución.....	4
1. Preparar el código fuente.....	4
2. Compilar y ejecutar el intérprete.....	5
Manejo de errores y excepciones.....	5
Errores sintácticos.....	5
División entre cero.....	5
Excepción de puntero nulo al buscar variable.....	5



Introducción

JavaRunner es un intérprete académico desarrollado en Java diseñado para leer, analizar, validar y ejecutar código fuente escrito en un subconjunto estructurado del lenguaje Java. El sistema procesa el código a través de fases de análisis léxico y sintáctico, generando una representación intermedia basada en estructuras de ejecución secuencial de tuplas apoyadas por una Tabla de Símbolos dinámica. Permitiendo al sistema procesar código que conforma la estructura de un programa escrito en el lenguaje de programación Java

Requisitos del sistema

Para compilar y ejecutar JavaRunner, el debe contar con:

- Java JDK 8 o superior instalado en el computador.
- Un editor de texto o IDE (VS CODE, Eclipse, etc.).
- Archivo fuente del proyecto.

Sintaxis y características permitidas

El intérprete valida y procesa las siguientes estructuras gramaticales:

Tipos de Datos y Variables

Tipo	Descripción
int	Valores enteros de 32 bits.
double	Valores decimales del tipo flotante.
String	Cadenas de texto delimitadas por comillas.

Operadores

Tipo	Simbología
Aritméticos	+, -, *, /, %
Relacionales	==, !=, <, >, <=, >=
Asignación e incremento	=, ++



Estructuras de Control

Tipo	Descripción
Condicionales	Bloques if, else.
Ciclos	Ciclos indeterminados while y bucles determinados for.

Salida estándar

Impresión de variables y literales mediante System.out.println(...).

Guia de uso y ejecución

1. Preparar el código fuente

El primer paso es crear nuestro archivo con extensión .java que contenga el código fuente. El cual debería tener la siguiente estructura:

```
Java
public class code {
    public static void main(String[] args) {
        int limite;
        double promedio;
        String mensaje;
        limite = 4;
        promedio = 0;
        mensaje = "Procesando ciclo ";

        for (int i = 1; i <= limite; i++) {
            promedio = promedio + i;
            System.out.println(mensaje + i);
        }
        promedio = promedio / limite;
        System.out.println("El promedio final es: " + promedio);

        if (promedio > 2.0) {
            System.out.println("Resultado satisfactorio");
        } else {
            System.out.println("Resultado bajo");
        }
    }
}
```



2. Compilar y ejecutar el intérprete

Dirígete a tu terminal de comandos dentro de la ruta raíz del proyecto y compila todos los módulos del proyecto.

None

```
javac *.java && java Main
```

Manejo de errores y excepciones

El sistema cuenta con un control estricto de anomalías en tiempo de análisis y ejecución. A continuación, se listan las alertas comunes y cómo solucionarlas:

Errores sintácticos

“Se esperaba Token... en línea n”.

- **Causa:** Falta un delimitador (como ;), un paréntesis de cierre “)” o una llave “{” en la instrucción indicada.
- **Solución:** Revisa la línea señalada en code.java y añadir el carácter faltante.

División entre cero

- **Causa:** Una instrucción aritmética de tipo división (/) o residuo (%) intentó procesar un denominador cuyo valor en la tabla de símbolos se evaluó en 0.
- **Solución:** Modifica la lógica de las asignaciones para asegurar que el divisor sea diferente de cero antes de realizar la operación.

Excepción de puntero nulo al buscar variable

- **Causa:** Se está intentando operar o asignar un valor utilizando una variable que no ha sido declarada previamente al inicio del bloque.
- **Solución:** Asegúrate de incluir la declaración explícita del tipo (int, double o String) antes de invocar por su identificador.